

Software Handbuch

SILworX Plug-In Asset_Inventory

```
#----- OPERATOR CLASSES -----
# Mirror Tool
class MirrorX(bpy.types.Operator):
    """This adds an X mirror to the selected object"""
    bl_idname = "object.mirror_mirror_x"
    bl_label = "Mirror X"

    @classmethod
    def poll(cls, context):
        return context.active_object is not None
        mirror_mod = modifier_ob.modifiers.new("Mirror", "MIRROR_X")

    # set mirror object to mirror_ob
    mirror_mod.mirror_object = mirror_ob

    if _operation == "MIRROR X":
        mirror_mod.use_x = True
        mirror_mod.use_y = False
        mirror_mod.use_z = False
    elif _operation == "MIRROR Y":
        mirror_mod.use_x = False
        mirror_mod.use_y = True
        mirror_mod.use_z = False
    elif _operation == "MIRROR Z":
        mirror_mod.use_x = False
        mirror_mod.use_y = False
        mirror_mod.use_z = True

    #selection at the end -add back the modifier
    mirror_ob.select= 1
    modifier_ob.select= 1
    bpy.context.select= 1
    print("Selected" + str(modifier_ob))
    #mirror_ob.select = 0
    #bpy.data.objects[selected_objects[0]].select = 1
    except:
        print("please select exactly two objects")

#----- OPERATOR CLASSES -----
# Mirror Tool
class MirrorX(bpy.types.Operator):
    """This adds an X mirror to the selected object"""
    bl_idname = "object.mirror_mirror_x"
    bl_label = "Mirror X"

    @classmethod
    def poll(cls, context):
        return context.active_object is not None
        mirror_mod = modifier_ob.modifiers.new("Mirror", "MIRROR_X")

    # set mirror object to mirror_ob
    mirror_mod.mirror_object = mirror_ob

    if _operation == "MIRROR X":
        mirror_mod.use_x = True
        mirror_mod.use_y = False
        mirror_mod.use_z = False
    elif _operation == "MIRROR Y":
        mirror_mod.use_x = False
        mirror_mod.use_y = True
        mirror_mod.use_z = False
    elif _operation == "MIRROR Z":
        mirror_mod.use_x = False
        mirror_mod.use_y = False
        mirror_mod.use_z = True

    #selection at the end -add back the modifier
    mirror_ob.select= 1
    modifier_ob.select= 1
    bpy.context.select= 1
    print("Selected" + str(modifier_ob))
    #mirror_ob.select = 0
    #bpy.data.objects[selected_objects[0]].select = 1
    except:
        print("please select exactly two objects")
```

Alle in diesem Handbuch genannten HIMA Produkte sind mit dem Warenzeichen geschützt. Dies gilt ebenfalls, soweit nicht anders vermerkt, für weitere genannte Hersteller und deren Produkte.

HIQuad®, HIQuad® X, HIMax®, HIMatrix®, SILworX®, XMR®, HICore® und FlexSILon® sind eingetragene Warenzeichen der HIMA Paul Hildebrandt GmbH.

Alle technischen Angaben und Hinweise in diesem Handbuch wurden mit größter Sorgfalt erarbeitet und unter Einschaltung wirksamer Kontrollmaßnahmen zusammengestellt. Bei Fragen bitte direkt an HIMA wenden. Für Anregungen, z. B. welche Informationen noch in das Handbuch aufgenommen werden sollen, ist HIMA dankbar.

Technische Änderungen vorbehalten. Ferner behält sich HIMA vor, Aktualisierungen des schriftlichen Materials ohne vorherige Ankündigungen vorzunehmen.

Für registrierte Kunden stehen die Produktdokumentationen im HIMA Extranet als Download zur Verfügung.

© Copyright 2023, HIMA Paul Hildebrandt GmbH

Alle Rechte vorbehalten.

Kontakt

HIMA Paul Hildebrandt GmbH

Postfach 1261

68777 Brühl

Tel.: +49 6202 709-0

Fax: +49 6202 709-107

E-Mail: info@hima.com

Revisions-index	Änderungen	Art der Änderung	
		technisch	redaktionell
1.0	Erstausgabe	X	X
1.1	Aktualisierte Ausgabe zu Asset_Inventory V1.1	X	X

Inhaltsverzeichnis

1	SILworX Plug-Ins (lizenzpflichtige Option)	5
2	Plug-Ins konfigurieren	6
2.1	SSL-Zertifikat und -Schlüssel erstellen	6
2.2	«settings.ini» vervollständigen	7
3	Plug-In-Registrierung	9
3.1	Vermeidung von Fehlern bei der Plug-In-Registrierung	9
4	Plug-Ins starten	10
4.1	Plug-In Hash-Dateien	10
4.2	«customplugins.json» vervollständigen	11
5	Funktionsweise des Plug-Ins «Asset_Inventory.exe»	12

1 SILworX Plug-Ins (lizenzpflichtige Option)

Zur Erweiterung der Funktionalität kann SILworX ab Version 14 mit Plug-Ins ausgestattet werden. Plug-Ins sind nicht-sichere Anwendungen, die mit SILworX interagieren, aber nicht in der SILworX Entwicklungsumgebung enthalten sind und auch nicht mit SILworX zertifiziert werden.

Mit Plug-Ins sind Anwender in der Lage, zusätzliche Funktionen nach eigenem Bedarf zu erstellen oder auch für Dritte bereitzustellen. Voraussetzung ist das Vorhandensein der erforderlichen Lizenzen. Anwenderseitig erstellte Plug-Ins sind mit der Plug-In Feature-Lizenz in Kombination mit der SILworX API-Lizenz verwendbar. Für von HIMA bereitgestellte, lizenzfreie Plug-Ins wird nur die SILworX Lizenz benötigt.

Plug-Ins laufen als eigene Prozesse in einem von SILworX getrennten Speicherbereich und interagieren mit SILworX über die Schnittstellen *SILworX API* und *WebSocket*. Plug-Ins lassen sich rückwirkungsfrei einbinden und wirken sich nicht negativ auf die sicheren SILworX Funktionen aus.

Da Plug-Ins keine integralen Bestandteile von SILworX sind, müssen sie in der Datei *settings.ini* konfiguriert werden. Dafür existiert in der *settings.ini* die Kategorie *[Plugin_Server]*. SILworX prüft beim Starten das Vorhandensein der registrierten Plug-Ins und stellt deren Funktionalität in SILworX bereit. SILworX startet die in der *settings.ini* unter *PluginNames* aufgelisteten Plug-Ins automatisch. Die Reihenfolge der Plug-In-Namen bestimmt die Abarbeitungspriorität der Plug-Ins im Falle gleichzeitiger Triggerungen.

Plug-Ins werten von SILworX bereitgestellte vordefinierte Trigger und anwenderspezifische Trigger aus und führen die vom Anwender gewünschten Funktionen aus. Die von Plug-Ins in SILworX ausgelösten Aktionen werden im Logbuch protokolliert. In der Statuszeile werden rechts neben den API-Informationen in einem eigenen Bereich Informationen zum Plug-In-Feature angezeigt.

Die gestarteten Plug-Ins haben keine eigene Benutzerverwaltung. Die Authentifizierung erfolgt durch das Öffnen des Projekts in SILworX. Die dabei erzeugte User-Session steht auch den Plug-Ins und der SILworX API zur Verfügung.

Wenn das Plug-In-Feature gestartet ist, können Sie im *Hilfe*-Menü von SILworX über die Menüfunktion *Plug-In-Informationen* das entsprechende Dialogfenster öffnen. Das Dialogfenster enthält die beiden Listen *Konfigurierte Plug-Ins* und *Registrierte Plug-Ins*. Zusätzlich wird angezeigt, mit welchen Aktionen ein Plug-In in SILworX verknüpft ist und welche Trigger aus SILworX bzw. nach SILworX registriert sind.

Zusammenfassung:

- Plug-Ins sind SILworX Erweiterungen, die von Anwendern selbst erstellt werden können.
- Plug-Ins basieren auf Programmen oder Scripten, ggf. mit Zusatzdateien, die in SILworX konfiguriert werden.
- Plug-Ins sind eigenständige Module, die in eigenen Prozessen laufen.
- Plug-Ins werden automatisch mit SILworX gestartet.
- Plug-Ins haben über die SILworX API-Funktionen Zugriff auf SILworX.
- Plug-Ins benötigen eine Plug-In-Feature-Lizenz und eine SILworX API-Lizenz.
- Plug-Ins können auf vordefinierte Trigger und benutzerdefinierte Trigger reagieren.
- Das Ausführen von Plug-Ins wird in der SILworX Statusleiste angezeigt.

2 Plug-Ins konfigurieren

Damit Sie SILworX Plug-Ins verwenden können, müssen Sie die gewünschten Plug-Ins auf einem für SILworX zugänglichen Laufwerk speichern und in der Datei *settings.ini* konfigurieren. Zusätzlich benötigen Sie eine Plug-In-Feature-Lizenz, eine SILworX API-Lizenz und ein SILworX API-Zertifikat. Zugriffe auf die SILworX API sind ausschließlich über SSL (TLS 1.2 oder TLS 1.3) möglich.

Die Verwendung von SSL setzt ein gültiges Zertifikat und den zugehörigen privaten Schlüssel voraus. Die Pfade zu diesen Dateien müssen in der *settings.ini* in der Kategorie *[API_Server]* eingetragen werden. Damit Plug-Ins in Verbindung mit der SILworX API verwendet werden können, muss der Konfigurationsparameter *Enabled* der Kategorie *[API_Server]* in der *settings.ini* auf TRUE gesetzt werden.

SILworX liest beim Start diverse Plug-In-Konfigurationsparameter aus der *settings.ini* ein, die in der Kategorie *[Plugin_Server]* zusammengefasst sind. Danach prüft SILworX die Integrität der registrierten Plug-Ins anhand von Hash-Codes. Ist das Ergebnis der Integritätsprüfung positiv, stellt SILworX die Funktionen der Plug-Ins bereit.

Um Plug-Ins in SILworX nutzen zu können, müssen die Namen der Plug-Ins in der Liste *PluginNames*, durch Kommata getrennt, vorhanden sein. Wenn mehrere Plug-Ins auf einen bestimmten Trigger reagieren, bestimmt die Reihenfolge in der *PluginNames*-Liste die Ausführungsreihenfolge der Plug-Ins.



Maximal können 254 Plug-Ins registriert werden.

Sollten in *PluginNames* Plug-In-Namen mehrfach vorkommen, so werden die Duplikate beim Einlesen der Plug-In-Konfiguration verworfen und SILworX meldet einen Fehler.



Die Plug-In-Namen werden in den Meta-Daten der Plug-Ins definiert. Sie können unterschiedlich zu den verwendeten Dateinamen der Plug-Ins sein.

Die in *PluginNames* gelisteten Plug-Ins müssen als ausführbare Programme (*.exe) vorliegen und zum Schutz vor unerwünschten Manipulationen muss jedes dieser Plug-Ins in der Hash-Datei *customplugins.json* konfiguriert sein.



SILworX installiert einige von HIMA bereitgestellte Plug-Ins und trägt deren Namen standardmäßig als *PluginNames* ein. Diese Liste können Sie an beliebiger Position um Ihre eigenen *PluginNames* ergänzen. Auch für die Verwendung der HIMA Plug-Ins muss der API-Server **Enabled** sein.

SILworX startet nach der Initialisierung alle in *PluginNames* konfigurierten Plug-Ins automatisch. Dazu müssen für die in *PluginNames* gelisteten Plug-Ins die passenden Hash-Codes in der *customplugins.json* existieren. Die Dateipfade werden ebenfalls der *customplugins.json* entnommen. SILworX gibt im Logbuch Meldungen aus, wenn Plug-Ins nicht gestartet werden können.

2.1 SSL-Zertifikat und -Schlüssel erstellen

Um die SILworX API nutzen zu können, benötigen Sie ein SSL-Zertifikat im PEM-Format, welches Sie selbst erstellen. Zu diesem Zertifikat benötigen Sie außerdem den zugehörigen privaten Schlüssel im PEM-Format, der mit dem RSA-Algorithmus ohne Passphrase erstellt wurde.

Zertifikat und Schlüssel müssen in der SILworX settings.ini eingetragen werden.



Gültigkeitsdauer des SSL-Zertifikats beachten!

Wenn Sie ein neues SSL-Zertifikat erzeugen, können Sie dessen Gültigkeitsdauer in Tagen definieren. HIMA empfiehlt, die Gültigkeitsdauer auf 365 Tage oder weniger zu beschränken. Einige Clients (z. B. aktuelle Browser) akzeptieren keine selbstsignierten Zertifikate mit Gültigkeitsdauer größer 365 Tage.

So erzeugen Sie ein gültiges Zertifikat:

- Wählen Sie in SILworX den Menüpunkt **Extras, SILworX-API Zertifikat erstellen**. Der Dialog *SILworX-API Zertifikat erstellen* öffnet sich.
- Geben Sie im Feld *Dateipfad - Zertifikat* den Pfad für das Zielverzeichnis und den Zertifikatsnamen ein oder verwenden Sie die Schaltfläche rechts neben dem Feld *Dateipfad - Zertifikat*, um das Zielverzeichnis und den Zertifikatsnamen auszuwählen.
- Geben Sie im Feld *Dateipfad - Schlüssel* den Pfad für das Zielverzeichnis und den Schlüsselnamen ein oder verwenden Sie die Schaltfläche rechts neben dem Feld *Dateipfad - Schlüssel*, um das Zielverzeichnis und den Schlüsselnamen auszuwählen.
- Füllen Sie bei Bedarf die optionalen Felder mit Ihren Daten aus.
- Bestätigen Sie Ihre Eingaben mit **Erstellen**.

2.2

«settings.ini» vervollständigen

Zum Konfigurieren der SILworX API und der Plug-Ins müssen Sie die nachfolgenden Einträge in der Datei *settings.ini* ergänzen.

- Öffnen Sie die Datei *settings.ini* im SILworX Programmdaten-Verzeichnis %ProgramData%\SILworX_vXX.X.0 Rxxxx\settings\, z. B. C:\ProgramData\SILworX_v14.0.0 R2855\settings\settings.ini, mit einem Texteditor.
- Suchen Sie in der Datei *settings.ini* nach dem Eintrag *[API_Server]*. Existiert dieser Eintrag nicht, fügen Sie ihn am Ende der Datei an.
- Unterhalb von *[API_Server]* werden die nachfolgend beschriebenen Konfigurationsparameter benötigt, die Sie bei Bedarf ergänzen müssen.
`Enabled=true`
`IpAddress=::1`
`ServerCertificate=Absoluter Pfad auf die Zertifikatsdatei`
`ServerPrivateKey=Absoluter Pfad auf die Schlüsseldatei`
- Suchen Sie in der Datei *settings.ini* nach dem Eintrag *[Plugin_Server]*. Existiert dieser Eintrag nicht, fügen Sie ihn am Ende der Datei an.
- Unterhalb von *[Plugin_Server]* werden die nachfolgend beschriebenen Konfigurationsparameter benötigt, die Sie bei Bedarf ergänzen müssen
`PluginNames=Liste der Namen der Plug-Ins`



API-Requests über localhost

Plug-Ins verwenden den Domainnamen *localhost* zur Kommunikation mit der SILworX API. Das Betriebssystem löst *localhost* zuerst nach IPv6 auf, bevor eine IPv4-*Localhost*-Adresse zum Tragen kommt. Dadurch kann es bei jedem API-Aufruf zu Verzögerungen bei der Ausführung im Bereich einiger Sekunden kommen. Als Abhilfe sollte der SILworX API-Server auf die IPv6-*Localhost*-Adresse konfiguriert werden. Dazu ist in der *settings.ini* der Parameter `IpAddress=::1` einzutragen.

Beispiel für die «settings.ini»

```
...  
[API_Server]  
Enabled=true  
IpAddress=::1  
ServerCertificate=C:\Plugins\Certificates\api_cert.pem  
ServerPrivateKey=C:\Plugins\Certificates\api_key.pem  
[Plugin_Server]  
PluginNames=hima.ssl_certificates_creator, asset_inventory
```


3 Plug-In-Registrierung

Mit der Registrierung, die nach dem erfolgreichen WebSocket-Verbindungsaufbau erfolgt, melden sich Plug-Ins bei SILworX mit ihrem Namen, den Metadaten und ihren gewünschten Triggern an. Dazu müssen die Plug-Ins mit ihren Namen in der *settings.ini* korrekt konfiguriert sein.

Plug-Ins registrieren sich nach ihrem Start mittels Nachrichten, die sie an SILworX senden. War die Registrierung erfolgreich, werden im Logbuch Meldungen mit den Dateipfaden der Plug-Ins ausgegeben. Zusätzlich werden die Hash-Codes der Plug-Ins angezeigt.

3.1 Vermeidung von Fehlern bei der Plug-In-Registrierung

War die Registrierung nicht erfolgreich, wird im Logbuch eine Fehlermeldung mit einem Hinweis auf die Ursache der fehlgeschlagenen Registrierung ausgegeben. Zur Fehlervermeidung beachten Sie die folgenden Hinweise:

- **Mehrfachregistrierung von Plug-Ins**

Im Konfigurationsparameter *PluginNames* der *settings.ini* dürfen keine doppelten Namen vorkommen.

- **Nichterlaubte Registrierung eines Plug-Ins**

Im Konfigurationsparameter *PluginNames* der *settings.ini* muss der in den Meta-Daten eines Plug-Ins definierte Plug-In Name aufgeführt sein. Dieser Plug-In Name muss auch dem Schlüssel *plugin_name* in der *customplugins.json* als Wert zugewiesen sein.

- **Maximale Anzahl von Plug-Ins**

Es können in SILworX maximal 254 Plug-Ins gleichzeitig registrieren werden.

- **Maximale Anzahl von Triggern**

Die maximale Anzahl von 20 Trigger-Einträgen für das *Extra-Menü* und 100 Trigger-Einträgen für die Summe aller Kontextmenüs darf nicht überschritten werden.

- **Falsche Formatierung der Hash-Datei**

Achten Sie auf die korrekte Anzahl der öffnenden und schließenden Klammern sowie der Anführungszeichen. Leerzeichen außerhalb von Zeichenketten sind nicht signifikant. Der Pfad zur Plug-In Datei muss Schrägstriche / oder doppelte Rückstriche \\ enthalten, einfache Rückstriche \ sind nicht zulässig.

4 Plug-Ins starten

Das Starten von Plug-Ins bedeutet, dass Plug-Ins zunächst in SILworX registriert werden, um mit SILworX interagieren zu können. Nach der erfolgreichen Plug-In-Registrierung befinden sich die Programme im Arbeitsspeicher des Rechners. Die eigentlichen Interaktionen mit SILworX und die Datenverarbeitung werden durch Trigger-Benachrichtigungen ausgelöst, die SILworX an die Plug-Ins sendet.

SILworX startet bei der Initialisierung einen Plug-In-Prozess für jedes der in der *settings.ini* unter *PluginNames* konfigurierten Plug-Ins, sofern die Plug-In-Daten in einer der Plug-In Hash-Dateien eingetragen sind. Existieren diese Daten nicht, gibt SILworX eine Meldung aus und startet das Plug-In nicht.

SILworX startet die in der *settings.ini* aufgeführten Plug-Ins automatisch. Beim Start eines Plug-Ins übergibt SILworX eine Kommandozeile mit einigen Parametern. Die Kommandozeile wird auch als Meldung im Logbuch ausgegeben.

Die gestarteten Plug-Ins haben keine eigene Benutzerverwaltung, die Benutzerauthentifizierung findet auf Projektebene statt. Durch das Öffnen eines Projekts mit Benutzerverwaltung authentifiziert sich der Anwender im SILworX Projekt. Die dabei erzeugte User-Session steht sowohl den Plug-Ins als auch der darin verwendeten SILworX API zur Verfügung.

4.1 Plug-In Hash-Dateien

Für die in der *settings.ini* unter *PluginNames* konfigurierten Plug-Ins prüft SILworX vor deren Registrierung die Echtheit der Plug-Ins. Die Prüfung erfolgt durch Vergleiche der aktuell berechneten Hash-Codes der Plug-Ins mit den vom Hersteller der Plug-Ins bereitgestellten Sollwerten, die zuvor in Hash-Dateien gespeichert wurden.

SILworX verwendet zwei unterschiedliche Hash-Dateien, eine für HIMA Plug-Ins und eine weitere für Anwender Plug-Ins. Die Hash-Datei für HIMA Plug-Ins enthält die Hash-Codes derjenigen Plug-Ins, die ohne Plug-In-Feature-Lizenz und ohne API-Lizenz genutzt werden können.

SILworX verwirft beim Einlesen der Plug-In-Konfiguration aus der *settings.ini* und den Hash-Dateien doppelt vorkommende Plug-In-Namen und meldet diese als Fehler.



Die Hash-Datei für HIMA Plug-Ins darf kundenseitig nicht verändert werden.

Die Hash-Datei *customplugins.json* enthält die Hash-Codes der Plug-Ins, für die eine Plug-In-Feature-Lizenz benötigt wird. Diese Hash-Datei muss kundenseitig angepasst werden. Die Datei enthält für jedes Plug-In einen separaten Datenblock und ist wie folgt aufgebaut:

```
{
  "plugins": [
    {
      "plugin_name": "Plug-In Name gemäß Meta-Daten des Plug-Ins 1",
      "plugin_path": "Absoluter Pfad auf das Plug-In 1",
      "plugin_hash": "Hash-Code nach SHA-256 Plug-In 1"
    },
    {
      "plugin_name": " Plug-In Name gemäß Meta-Daten des Plug-Ins 2",
      "plugin_path": "Absoluter Pfad auf das Plug-In 2",
      "plugin_hash": "Hash-Code nach SHA-256 Plug-In 2"
    }
  ]
}
```

4.2 «customplugins.json» vervollständigen

Zum Konfigurieren der Plug-Ins müssen Sie die nachfolgenden Einträge in der Datei *customplugins.json* ergänzen.

- Öffnen Sie die Datei *customplugins.json* im SILworX Programmdatei-Verzeichnis %ProgramData%\SILworX_vXX.X.0 Rxxxx\plugins\, z. B. C:\ProgramData\SILworX_v14.0.0 R2855\plugins\customplugins.json, mit einem Texteditor.
- Die Hash-Datei enthält für jedes Plug-In einen separaten Datenblock, bestehend aus einer in geschweiften Klammern eingeschlossenen Liste von Schlüssel : Werte-Paaren:

```
{  
  "plugin_name": "Plug-In Name gemäß den Meta-Daten des Plug-Ins",  
  "plugin_path": "Absoluter Pfad auf das Plug-In",  
  "plugin_hash": "Hash-Code nach SHA-256"  
}
```

Ergänzen Sie in der Datei *customplugins.json* diesen Datenblock mit den für das Plug-In passenden Werten. Existiert dieser Datenblock nicht, fügen Sie ihn nach der öffnenden eckigen Klammer ein. Beachten Sie, dass die Zeichenketten in doppelt geraden Anführungszeichen " gesetzt werden müssen.



Rückstriche \ in JSON-Zeichenketten

Entsprechend der JSON-Formatdefinition wird der Rückstrich \ in JSON-Zeichenketten als Maskierungszeichen zur Einleitung von Escape-Sequenzen verwendet. Daher müssen die Rückstriche in Pfadangaben entweder durch doppelte Rückstriche \\ oder durch Schrägstriche / ersetzt werden.

Beispiel für die «customplugins.json»

```
{  
  "plugins": [  
    {  
      "plugin_name": "asset_inventory",  
      "plugin_path": "C:/Plugins/Asset_Inventory.exe",  
      "plugin_hash":  
        "C6D68C64CAE01490E1AE102F4DA12AA19C705FA85FC44C89082C03104FDCFE20"  
    }  
  ]  
}
```

5 Funktionsweise des Plug-Ins «Asset_Inventory.exe»

Das Plug-In wird über das Kontextmenü einer Ressource gestartet:

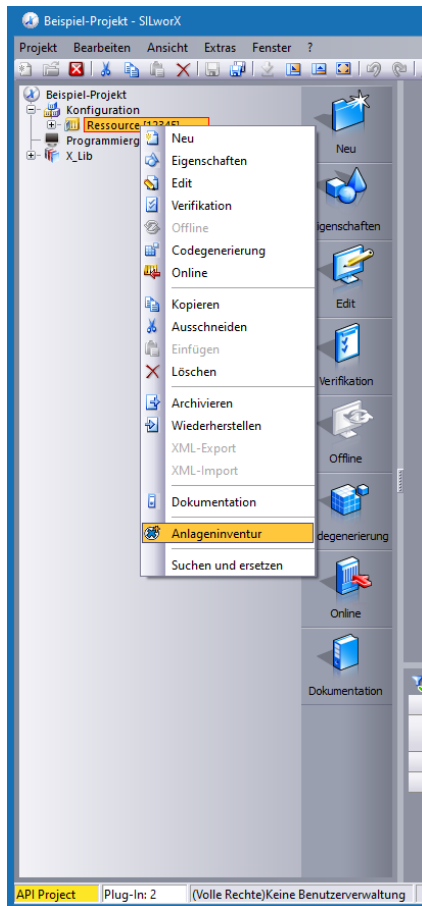


Bild 1: Resource-Kontextmenü mit Menüeintrag für das Plug-In

Das Plug-In liest Daten aus SILworX, dem geöffneten Projekt sowie der laufenden Steuerung aus und speichert die gesammelten Daten anschließend zur weiteren Verarbeitung in einer CSV-Datei.

Dazu führt es nacheinander die folgenden SILworX-Aktionen aus:

- Abrufen von Produktname, Version und Lizenz aus dem Menü **Hilfe, Info über SILworX**
- Abrufen des Projektnamens aus dem Strukturbaum
- Abrufen der System-ID aus dem Menü **Extras, Eigenschaften** der markierten Ressource
- Ausführen eines System-Logins mit Zugriffsart „Lesen“ über das Menü **Extras, Online** der markierten Ressource
- Abrufen der hauptsächlichen System- und Statusinformationen aus dem Control Panel der verbundenen Ressource
- Abrufen der Hardware-Modulinformationen über das Menü **Online, Moduldaten-Übersicht** der verbundenen Ressource
- Trennen der Online-Sitzung der verbundenen Ressource über das Menü **Online, Trennen**

Während der Datenaufnahme zeigt das Plug-In über eine Meldung die Ausführung der Aktionen.

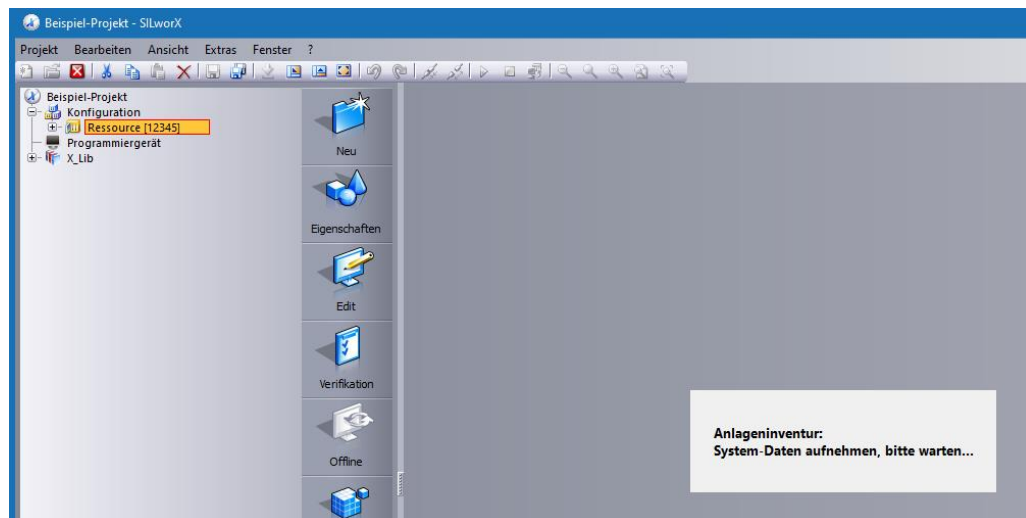


Bild 2: Rückmeldung des Plug-Ins

Besitzt das SILworX-Projekt eine Benutzerverwaltung, kann das Plug-In keinen System-Login mit Standard-Zugangsdaten durchführen. In diesem Fall fragt das Plug-In über einen Dialog die Zugangsdaten zur Steuerung ab.

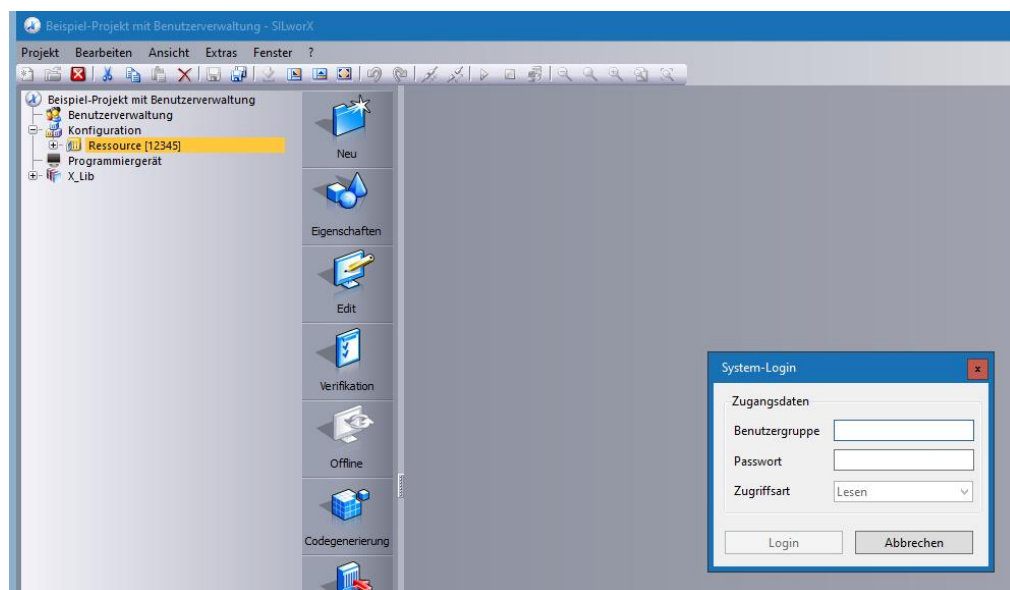


Bild 3: Anmeldedialog des Plug-Ins für den System-Login

Nachdem alle Inventardaten abgerufen sind, erscheint ein weiteres Dialogfenster zur Sicherung der Daten im CSV-Format.

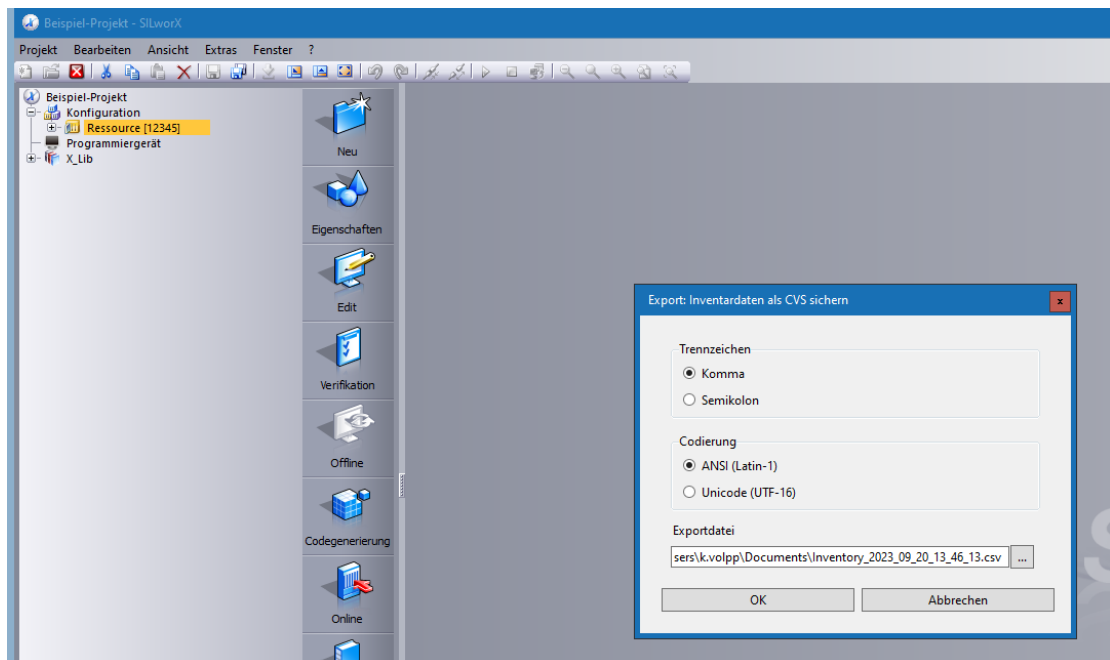


Bild 4: Dialogfenster des Plug-Ins zur Sicherung der Inventardaten

Als Standardeinstellung wird die CSV-Datei im Benutzerverzeichnis %UserProfile%\Documents\ unter dem Namen «Inventory_yyyy_mm_dd_hh_mm_ss.csv» abgelegt, wobei «yyyy_mm_dd_hh_mm_ss» dem Datum und der Uhrzeit des Starts des Plug-Ins entsprechen. Die CSV-Datei enthält die nachfolgend beispielhaft dargestellten Informationen in tabellarischer Form, die mit einem beliebigen Tabellenkalkulationsprogramm oder Datenbanksystem weiter verarbeitet werden können.

	A	B	C	D	E	F	G	H	I
1	Datum	Uhrzeit							
2	13.06.2023	16:07:50.201							
3									
4	Produkt	Version	Lizenz	Gültig bis					
5	SILworX	14.0.0 R2547	ID_Locked (10) Lizenz	unbegrenzt					
6									
7	Projekt	Konfiguration	Ressource	System-ID					
8	CSC-Demo	Konfiguration	Demo_BMS	100					
9									
10	System: Zustand	Status	Zeit	Betriebsdauer	Forcen	Zykluszeit: Mittlere	Minimale	Maximale	Systemfehler: Aktue
11	RUN	Fehler	13.06.2023 16:07	T#557d11h6m33s598ms	AKTIV	43	37	193	
12									
13	Modul: SRS	Typ	Name	Betriebssystem	Betriebssystem-Lader	Urlader	Hardware-Revision	Serien-Nummer	Adresse: MAC
14	SRS:100.0.1	X-SB 01	X-SB 01_1	V7.24	V6.0	V1.0	Rev.03	SN: 98501020701119973022	00-e0-a1-00-fd-6c
15	SRS:100.0.2	X-SB 01	X-SB 01_1	V7.24	V6.0	V1.0	Rev.03	SN: 98501020701119973023	00-e0-a1-00-fd-61
16	SRS:100.0.3	X-CPU 01	X-CPU 01_1	V13.2	V6.0	V1.0	Rev.01	SN: 98501021101118973008	00-e0-a1-00-cd-8e
17	SRS:100.0.4	X-COM 01	X-COM 01_1	V14.8	V6.0	V1.0	Rev.00	SN: 98506003100120812016	00-e0-a1-00-d5-1e
18	SRS:100.0.5	X-COM 01	X-COM 01_1	V14.8	V6.0	V1.0	Rev.00	SN: 98506003100120812013	00-e0-a1-00-d5-21
19	SRS:100.0.6	X-AO 16 01	X-AO 16 01_1	V7.20	V6.0	V1.0	Rev.10	SN: 985210210001000274222010	
20	SRS:100.0.7	X-HART 32 01	X-HART 32 01_1	V7.56	V6.0	V2.6	Rev.10	SN: 1000852029	
21	SRS:100.0.8	X-DI 32 01	X-DI 32 01_1	V7.20	V6.0	V1.0	Rev.02	SN: 98501020101120582032	
22	SRS:100.0.9	X-DO 32 01	X-DO 32 01_1	V7.28	V6.0	V2.6	Rev.12	SN: 985210219001201266061057	
23	SRS:100.0.10	X-CI 24 01	X-CI 24 01_1	V7.28	V6.0	V2.6	Rev.11	SN: 985210220001101275824002	
24	SRS:100.0.11	X-AO 16 01	X-AO 16 01_1	V7.52	V6.0	V1.0	Rev.02	SN: 98521021010144594014	
25	SRS:100.0.12	X-MIO 7/6 01	X-MIO 7/6 01_1	V7.48	V6.0	V2.6	Rev.10	SN: 985210238001002277543002	
26	SRS:100.0.13	X-MIO 7/6 01	X-MIO 7/6 01_1	V7.48	V6.0	V2.6	Rev.10	SN: 985210238001002277543001	
27	SRS:100.0.14	X-DI 32 01	X-DI 32 01_1	V7.34	V6.0	V1.0	Rev.02	SN: 98501021901117450008	

Bild 5: Ausschnitt eines Beispiels mit den Inventardaten der CSV-Datei

Das Plug-In erzeugt zusätzlich im Verzeichnis %AppData%\SILworX_Plugins\ eine Logdatei mit dem Namen «asset_inventory.log». Die Logdatei enthält alle aufgetretenen Status- und Fehlermeldungen mit Angabe des Zeitpunkts, zu dem die Meldung ausgegeben wurde. Die Ausgabe erfolgt in tabellarischer Form.

Anhang

Glossar

Begriff	Beschreibung
SILworX	Programmierwerkzeug für HIMax, HIQuad X und HIMatrix Steuerungen
API	Application Programming Interface, Programmierschnittstelle
HTTPS	Hypertext Transfer Protocol Secure, Kommunikationsprotokoll
SSL	Secure Sockets Layer, alte Bezeichnung für TLS
TLS	Transport Layer Security, Verschlüsselungsprotokoll
PEM	Privacy Enhanced Mail, Dateiformat nach RFC 7468
RSA	Rivest–Shamir–Adleman, asymmetrisches kryptographisches Verfahren
localhost	localhost benennt in der Netzwerktechnik den lokalen Rechner
SHA	Secure Hash Algorithm, kryptologische Hashfunktion
JSON	JavaScript Object Notation, Datenformat nach RFC 8259
CSV	Comma-Separated Values, Dateiformat nach RFC 4180

Abbildungsverzeichnis

Bild 1:	Resource-Kontextmenü mit Menüeintrag für das Plug-In	12
Bild 2:	Meldungsfenster des Plug-Ins mit Fortschrittsbalken	13
Bild 3:	Meldungsfenster des Plug-Ins mit Aufforderung zum System-Login	13
Bild 4:	Dialogfenster des Plug-Ins zur Sicherung der Inventardaten	14
Bild 5:	Ausschnitt eines Beispiels mit den Inventardaten der CSV-Datei	14

HANDBUCH
SILworX-Plug-In
Asset_Inventory

Für weitere Informationen kontaktieren Sie:

HIMA Paul Hildebrandt GmbH
Albert-Bassermann-Str. 28
68782 Brühl, Germany

Telefon +49 6202 709-0
Fax +49 6202 709-107
E-Mail info@hima.com

Erfahren Sie online mehr über HIMA Lösungen:

 www.hima.com/de/



www.hima.com